MMUKO-OS Byzantine Hardware Fault-Tolerance Specification

1. Definition

MMUKO-OS Byzantine Hardware Fault Tolerance, abbreviated **BHFT**, is a hardware resilience model in which physical components may fail arbitrarily while the operating system continues to preserve verifiable system behaviour.

A Byzantine hardware component may:

- stop responding;
- return incorrect data;
- return different data to different observers;
- report a false health state;
- execute an instruction incorrectly;
- claim that repair or verification succeeded when it failed;
- behave correctly intermittently;
- corrupt connected components;
- violate timing, electrical, logical, structural, or security invariants;
- continue operating while physically damaged.

MMUKO-OS shall therefore never trust a component solely because the component reports that it is functional.

$$\text{Self-reported health} \ / \ \text{verified health} \qquad =$$

---

2. Byzantine Hardware Model

Let the MMUKO-OS hardware system be represented as:

$$G = (V, E)$$

where:

- V is the set of hardware components;
- E is the set of communication, power, timing, thermal, mechanical, material, and trust relationships.

Each component $H_i \in V$ has the state:

$$H_i = (I_i, S_i, O_i, F_i, C_i, V_i)$$

where:

- I_i is the component identity;
- S_i is its physical lifecycle state;

- O_i is its observed output;
- F_i is its active fault set;
- C_i is its confidence score;
- V_i is its verification state.

A component is Byzantine when its observable behaviour cannot be assumed to follow its declared specification:

$$H_i \in B \iff O_i \not\models Spec(H_i)$$

or when insufficient trusted evidence exists to prove that it satisfies the specification:

$$Evidence(H_i) < \theta_{\text{safe}}$$

---

3. Fault Classes

MMUKO-OS shall distinguish the following fault classes:

```
typedef enum mmuko_fault_class {
    MMUKO_FAULT_NONE = 0,
    MMUKO_FAULT_CRASH,
    MMUKO_FAULT_OMISSION,
    MMUKO_FAULT_TIMING,
    MMUKO_FAULT_TRANSIENT,
    MMUKO_FAULT_DEGRADED,
    MMUKO_FAULT_CORRUPTION,
    MMUKO_FAULT_CONTRADICTORY,
    MMUKO_FAULT_DECEPTIVE,
    MMUKO_FAULT_COMPROMISED,
    MMUKO_FAULT_PHYSICAL,
    MMUKO_FAULT_PROPAGATING,
    MMUKO_FAULT_BYZANTINE,
    MMUKO_FAULT_UNKNOWN
} mmuko_fault_class_t;
```

Crash fault

The component stops operating.

Omission fault

The component fails to produce a required result.

Timing fault

The component produces a result outside the permitted timing interval.

Corruption fault

The component produces malformed or incorrect data.

Contradictory fault

The component produces inconsistent outputs to different observers.

Deceptive fault

The component reports false state, evidence, health, or repair results.

Physical fault

The material, electrical, thermal, mechanical, or structural state violates the component specification.

Propagating fault

The malfunction can spread through power, heat, data, mechanical, material, or control connections.

Byzantine fault

The component may exhibit any combination of the above behaviours without a predictable failure pattern.

---

4. Verification States

MMUKO-OS shall use three-state verification:

```
typedef enum mmuko_verify_state {
    MMUKO_VERIFY_NO = 0,
    MMUKO_VERIFY_YES = 1,
    MMUKO_VERIFY_MAYBE = 2
} mmuko_verify_state_t;
```

The states mean:

$$YES = \text{independent evidence confirms conformance}$$

$$NO = \text{evidence confirms specification violation}$$

$$MAYBE = \text{evidence is incomplete, conflicting, stale, or untrusted}$$

A component classified as `MAYBE` shall not be assumed safe.

$$MAYBE \ / \ YES \qquad =$$

A component shall not verify itself as the sole source of evidence.

---

5. Byzantine Quorum

For replicated hardware components, MMUKO-OS may use a Byzantine quorum.

Under the classical Byzantine model:

$$n \geq 3f + 1$$

where:

- n is the total number of independent replicas;
- f is the maximum number of Byzantine replicas that may be tolerated.

Agreement requires at least:

$$2f + 1$$

consistent responses.

For example, to tolerate one arbitrary faulty component:

$$f = 1$$

$$n \geq 4$$

At least three independently verified responses are required for strong agreement.

MMUKO-OS shall not count replicas as independent when they share the same:

- power source;
- clock source;
- firmware image;
- verification sensor;
- physical substrate;
- communication path;
- manufacturing defect;
- controlling process.

Shared failure domains reduce effective redundancy.

---

6. Physical Consensus

Hardware agreement shall not depend only on digital output comparison.

A component result may require multiple evidence channels:

$$Consensus(H_i) = L_i \wedge E_i \wedge T_i \wedge P_i$$

where:

- L_i is logical verification;
- E_i is electrical verification;
- T_i is timing verification;
- P_i is physical or structural verification.

For a processor operation, evidence may include:

- output comparison;
- timing bounds;
- voltage and current measurements;
- thermal measurements;
- instruction trace;
- memory integrity;
- independent recomputation.

A matching result from two components is not sufficient when both components may share the same fault source.

---

7. Byzantine Hardware Lifecycle

The MMUKO-OS lifecycle is:

$$\mathrm{CREATE} \to \mathrm{BUILD} \to \mathrm{VERIFY} \to \mathrm{ACTIVE}$$

A fault may produce:

$$\mathrm{ACTIVE} \to \mathrm{SUSPECT} \to \mathrm{DEGRADED} \to \mathrm{BYZANTINE}$$

Containment proceeds as:

$$\mathrm{BYZANTINE} \to \mathrm{ISOLATE} \to \mathrm{DIAGNOSE}$$

Recovery may then select:

$$\mathrm{REPAIR} \vee \mathrm{RENEW} \vee \mathrm{REPLACE} \vee \mathrm{RECONSTRUCT} \vee \mathrm{DESTROY}$$

Every recovery path must return through verification:

$$\mathrm{RECOVERY} \to \mathrm{VERIFY} \to \left\{ \mathrm{ACTIVE}, \quad YES\ \mathrm{QUARANTINED}, \ MAYBE\ \mathrm{BROKEN}, \quad NO \right.$$

---

8. Independent Verification Rule

A mutation actuator shall not be the sole verifier of its own operation.

$$Actor_{\mathrm{mutation}} \ / \ Actor_{\mathrm{verification}} =$$

For example, a nanoscale repair controller that modifies a circuit shall not independently declare that circuit safe without external evidence.

Verification should use:

- independent sensors;
- redundant computation;
- structural checks;
- cryptographic attestation;
- known test vectors;
- challenge-response testing;
- physical measurement;
- temporal observation.

---

9. On-the-Fly Malfunction Handling

MMUKO-OS shall support faults that occur during instruction execution or physical mutation.

Let:

$$\delta(H, O, E) \to H'$$

represent a hardware operation.

A malfunction may occur at any intermediate point:

$$H \xrightarrow{\text{operation}} H_1 \xrightarrow{\text{fault}} H_B$$

where H_B is a Byzantine or physically unverified state.

MMUKO-OS shall therefore divide physical operations into observable checkpoints:

$$O = o_1, o_2, \ldots, o_k$$

After every safety-critical checkpoint:

$$Verify(H_j) \in \{YES, NO, MAYBE\}$$

Execution shall stop when verification returns `NO`.

Execution shall pause, isolate, or enter a safe state when verification returns `MAYBE`.

---

10. Byzantine Repair Transaction

A repair transaction shall follow:

$$\text{DECLARE} \to \text{AUTHORISE} \to \text{ISOLATE} \to \text{DIAGNOSE} \to \text{REPAIR} \to \text{CHALLENGE} \to \text{VERIFY} \to \text{CO}$$

The repair is not trusted merely because the repair device reports completion.

```
typedef struct mmuko_repair_result {
    uint64_t transaction_id;
    uint64_t component_id;

    bool actuator_reported_success;
    bool independent_verification_passed;
    bool quorum_reached;
    bool fault_removed;
    bool new_fault_detected;

    mmuko_verify_state_t verification;
} mmuko_repair_result_t;
```

The following condition is prohibited:

```
actuator_reported_success == true &&
independent_verification_passed == false
```

from producing an `ACTIVE` state.

---

11. Malfunction Evidence

Every fault report shall include provenance:

```
typedef struct mmuko_fault_evidence {
    uint64_t evidence_id;
    uint64_t observer_id;
    uint64_t component_id;

    uint64_t timestamp_ns;
    uint64_t measurement_type;

    double measured_value;
    double confidence;

    uint64_t evidence_hash;
    uint64_t signature;
} mmuko_fault_evidence_t;
```

MMUKO-OS shall consider:

- whether observers agree;
- whether observers are independent;
- whether evidence is recent;

- whether sensors may themselves be Byzantine;
- whether the fault is transient;
- whether the evidence can be reproduced;
- whether the result violates a physical invariant.

---

12. Fault Propagation

A Byzantine hardware fault may spread through the component graph.

For an edge E_{ij}:

$$P(F_j \mid F_i, E_{ij})$$

represents the probability that fault F_i in component H_i causes fault F_j in component H_j.

Propagation channels include:

- power;
- heat;
- electromagnetic interference;
- data corruption;
- clock corruption;
- mechanical stress;
- material contamination;
- malicious firmware;
- faulty repair instructions.

A Byzantine component shall be isolated before repair when continued connectivity could spread the fault.

---

13. C and Assembly Semantics

The C and assembly interfaces shall expose hardware uncertainty explicitly.

```
typedef enum mmuko_execution_status {
    MMUKO_EXEC_DEFINED = 0,
    MMUKO_EXEC_FAILED,
    MMUKO_EXEC_UNVERIFIED,
    MMUKO_EXEC_BYZANTINE,
    MMUKO_EXEC_QUARANTINED
} mmuko_execution_status_t;
```

A C operation that is valid according to the C abstract machine may still fail physically:

$$Defined_C(op) \not\Rightarrow Verified_{hardware}(op)$$

Similarly, a valid assembly instruction may produce an untrusted result when the processor, memory, bus, or clock is faulty.

Therefore:

$$ProgramResult = Value + Evidence + VerificationState$$

rather than only:

$$ProgramResult = Value$$

---

14. Non-Negotiable Invariants

$$\text{Self-report} \not\Rightarrow \text{trust}$$

$$\text{Matching outputs} \not\Rightarrow \text{independent agreement}$$

$$\text{Repair completed} \not\Rightarrow \text{repair verified}$$

$$\text{Component responsive} \not\Rightarrow \text{component correct}$$

$$\text{MAYBE} \not\Rightarrow \text{safe}$$

$$\text{BYZANTINE} \rightarrow \text{ISOLATE before mutation}$$

$$\text{BROKEN} \not\rightarrow \text{ACTIVE without independent verification}$$

---

15. Summary

MMUKO-OS Byzantine Hardware Fault Tolerance treats hardware malfunction as an arbitrary-failure problem rather than only a crash or repair problem.

A faulty hardware component may lie, disagree, corrupt data, violate timing, report false health information, or behave correctly only intermittently.

MMUKO-OS preserves trust by combining:

- independent observation;
- fault-domain separation;
- replicated execution;
- Byzantine quorum;
- physical evidence;
- component isolation;
- bounded repair;
- renewal and reconstruction;
- post-repair verification;
- `YES / NO / MAYBE` consensus.

The central principle is:

Do not trust hardware behaviour; verify hardware behaviour.